DES PUNE UNIVERSITY
School of Engineering and Technology
Computer Engineering and Technology
Program: B.Tech. Computer Science and Engineering

Academic Year: 2025-26	Year: Second Year	Term: I
Roll No.: 1012411011	Name: Riddhee Kulkarni	
Subject: Java		
Assignment No.: 11	Title: Client-Server application	
Date: 08/10/2025		

Aim:

Develop a simple client-server application using Socket and ServerSocket classes for basic communication.

Theory:

Server Side:

On the server side, Java uses the ServerSocket class to establish a server that listens for incoming connections from clients. The server creates a ServerSocket object on a specific port number and waits for clients to connect using the accept() method. This method blocks the program until a client connects, after which it returns a Socket object that enables communication with that particular client. The server can then use input and output streams to receive data from and send data to the client. Once the communication is complete, the server can close the connection or continue to listen for more clients.

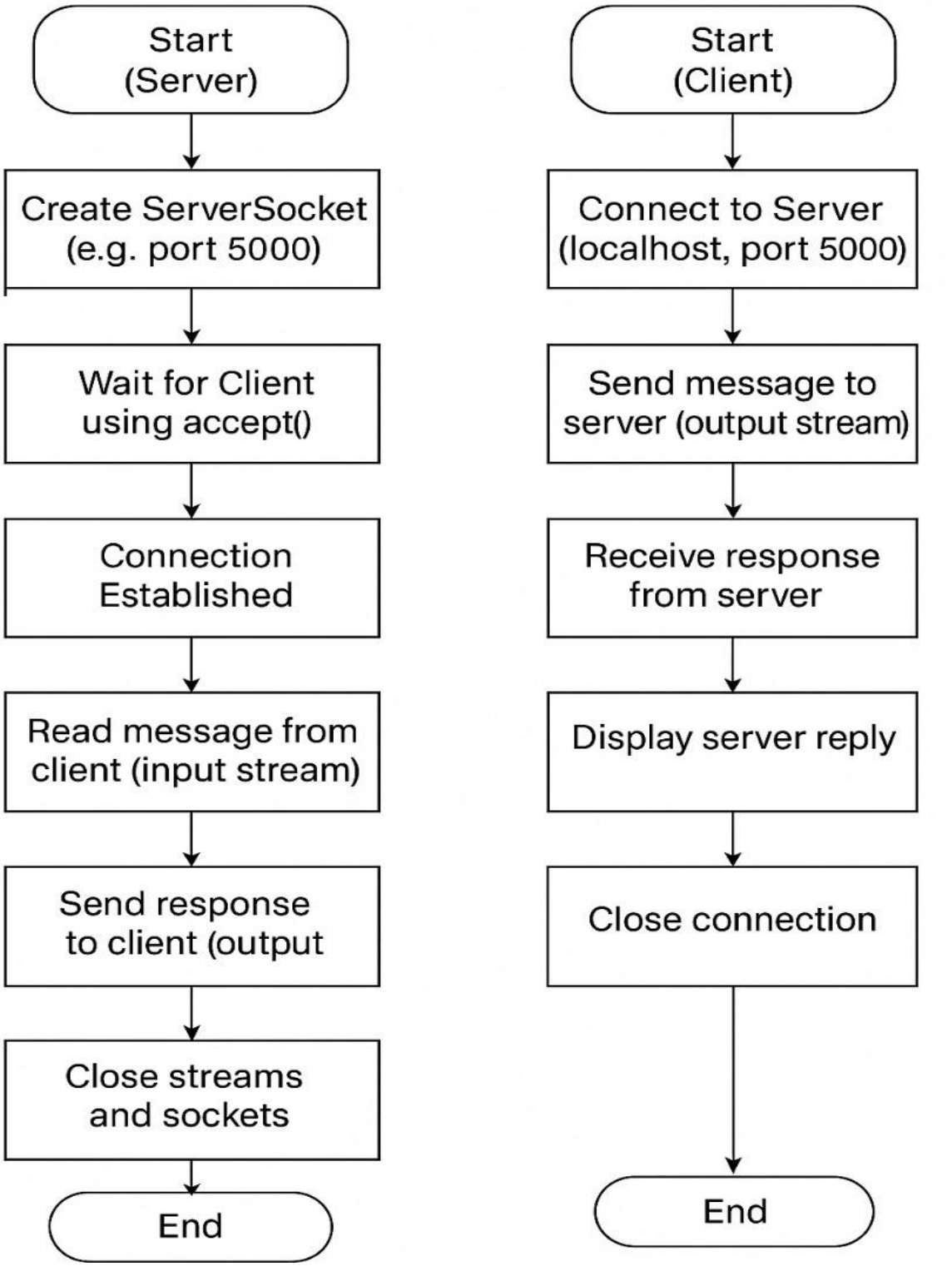
Client Side:

The client side uses the Socket class to connect to the server. The client creates a `Socket` object by specifying the server's IP address (or hostname) and the port number on which the server is listening. Once the connection is established, the client can communicate with the server using input and output streams. The client sends data or requests to the server and receives responses in return. This setup allows the client to access resources or services provided by the server, such as retrieving information, sending messages, or performing specific operations.

Server-Client Application:

A **client-server application** in Java is a network-based program where the client and server communicate through sockets. The server continuously listens for connection requests using `ServerSocket`, and the client initiates the connection using `Socket`. After the connection is established, both can exchange data bidirectionally using streams. This model is widely used in real-world applications such as chat systems, online games, web servers, and database connections, where multiple clients interact with a centralized server to share or request information.

Flowchart:


Start (Server)
Create ServerSocket (e.g. port 5000)
Wait for Client using accept()
Connection Established
Read message from client (input stream)
Send response to client (output
Close streams and sockets
End
Start (Client)
Connect to Server (localhost, port 5000)
Send message to server (output stream)
Receive response from server
Display server reply
Close connection
End

Code:

Server:

```
package Assignment11;

import java.io.*;
import java.net.*;

public class Server
{
    public static void main(String[] args)
    {
        try
        {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server is waiting for client...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");

            BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String message = input.readLine();
            System.out.println("Message from client: " + message);

            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
            output.println("Hello Client, message received!");

            socket.close();
            serverSocket.close();
        }

        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Client:

```
package Assignment11;

import java.io.*;
import java.net.*;
public class Client
{
    public static void main(String[] args)
    {
        try
        {
            Socket socket = new Socket("localhost", 5000);

            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
            output.println("Hello Server!");

            BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String reply = input.readLine();
            System.out.println("Message from server: " + reply);

            socket.close();
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output:

Server:

```
Server is waiting for client...
Client connected!
Message from client: Hello Server!
```

Client:

```
Message from server: Hello Client, message received!
```

Conclusion:

A simple client-server application using Socket and ServerSocket classes for basic communication is developed.